

SIU Fraud Investigation

1. Executive summary

Fraud investigators today open a reactive tool, paste a claim number, and read dozens of signals before deciding whether a referral is real. This prototype flips that: when the investigator opens the queue, **every case has already been assessed by an AI** with a risk level, a confidence level, a short plain-English overview, the exact signals that drove the conclusion, and a recommended next step. The investigator stays in control: raw evidence sits next to the AI's reasoning, follow-up questions are answered in natural language, and accept / escalate / dismiss is one click plus a note.

The build is a focused foundation: **Python/FastAPI backend, PostgreSQL 16, LangChain + Google Gemini (gemini-3.6-flash) with strictly structured output, and a React + TypeScript UI**. This write-up follows the SDLC: requirements, design, build, trade-offs, and the road to scale.

2. Requirements gathering

Who the user is and the job to be done

The user is a Special Investigations Unit (SIU) fraud investigator working through a daily queue of claims produced by a rules + ML fraud engine. Their job-to-be-done is **“decide quickly and consistently which referrals deserve my time, and be able to justify that decision.”** The pain is that benign cases consume the most time and conclusions vary between investigators.

Functional requirements (from the brief) and how they were scoped

Requirement	Decision for the prototype	Why
Ingest a queue of referred claims	Load the 50-row CSV into Postgres on startup, idempotent so restarts never re-score cases	Mirrors a nightly batch feed, avoids burning LLM calls twice
Reason over each case	One grounded LLM call per batch of 10 cases, with a system prompt that forbids inventing facts	Simplest design that is explainable, multi-agent adds cost without adding evidence
Per-case assessment before a human looks	Risk (High/Med/Low), confidence (High/Med/Low), 2–3 sentence overview, 1–4 cited key drivers, next step	Confidence is a first-class output so uncertainty is shown, not hidden
Investigator interaction	Drill-in, grounded chat with memory, Accept-AI / Escalate / Dismiss, free-text notes	Covers all four interaction types in the brief in one screen
Out of scope (deliberately)	Auth, multi-tenancy, deployment pipeline, public-data enrichment, pagination	Focused on building the foundational logic

3. Design: architecture and schema

The system has two halves that share one database: an offline **proactive triage** pipeline, and an interactive **human-in-the-loop** review surface. Think of a junior analyst who writes a one-page brief on every file overnight, then sits beside the senior investigator in the morning to answer questions.

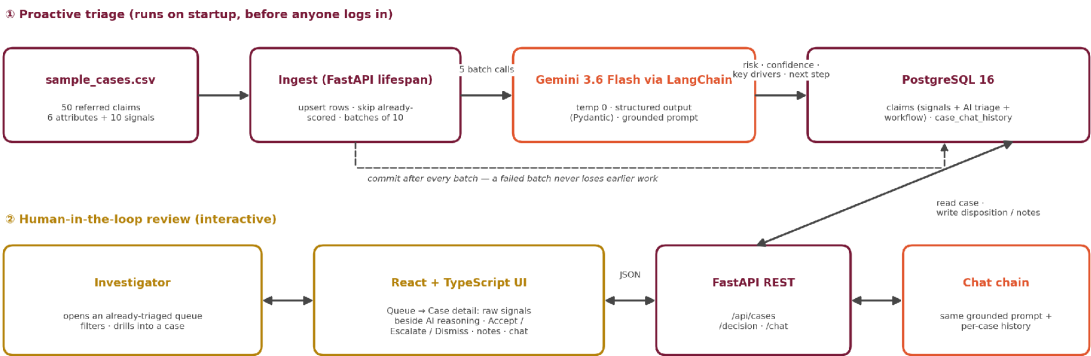


Figure 1 — End-to-end flow. Triage happens before the investigator arrives, review is interactive and every action is persisted.

Schema design

A single `claims` table keeps the three life-stages of a case side by side: what came in, what the AI concluded, and what the human decided, so one row tells the whole story and a future audit can compare AI output with the final disposition. Chat transcripts live in a separate table because they are append-only and unbounded.

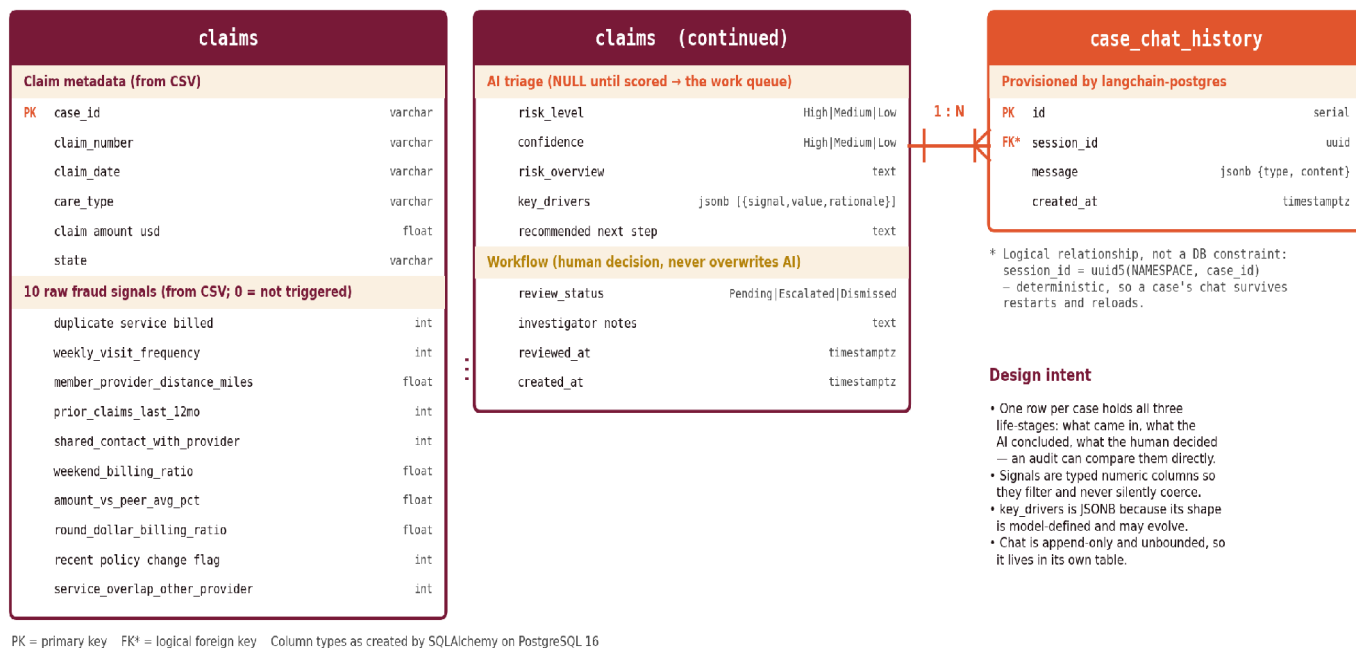


Figure 2 — Entity-relationship diagram

4. Development

How the AI reasons and how it is kept honest

Grounding is enforced at three layers rather than hoped for in a prompt.

1. **Input:** the model is shown only the 16 fields of each claim, rendered as plain text, no external context, no tools, no retrieval, so there is nothing to hallucinate from.
2. **Instruction:** the system prompt makes the model a senior SIU reviewer who must cite the exact field name and value behind every key driver, must lower its confidence when signals are few, weak, or conflicting, and must say so plainly.
3. **Output:** responses are forced into a Pydantic schema via LangChain structured output: risk_level and confidence are literal enums, key_drivers is a typed list, so a malformed answer fails loudly instead of rendering as garbage.

Triage runs in the FastAPI lifespan hook: rows are upserted, then every case with a NULL risk_level is scored in batches of 10 (five calls for the full queue). The database is committed after each batch, so a quota error or timeout on batch four never loses batches one to three, and a re-run only scores what is still missing. The chat endpoint reuses the same grounding rules plus a per-case message history, so “why is weekend billing flagged?” is answered from that claim's numbers, and the model must admit when the data cannot answer.

What the investigator sees (human in the loop)

The queue opens with High / Medium / Low counts, a risk filter and search. The case screen puts the **raw signals on the left** (values outside sensible thresholds are highlighted in amber) and the **AI assessment on the right**, so a reader can verify every key driver against the source number without leaving the page. Confidence is shown as a badge beside risk. Three buttons make the decision explicit: **Accept AI Recommendation** (maps the recommended step to Escalate or Dismiss), **Escalate to SIU**, and **Dismiss (False Positive)**. Notes and the chat transcript are saved per case. The AI never changes a case's status only a person does.

Engineering choices

- **Stack:** FastAPI + SQLAlchemy + psycopg, React 19 + TypeScript + Tailwind 4 (Vite), LangChain with langchain-google-genai and langchain-postgres. Postgres runs in one Docker container, app processes run natively for a fast iteration loop.

- **REST API:**

GET /api/cases : retrieve all the cases

GET /api/cases/{id} : retrieve the case with specific id

POST /api/cases/{id}/decision : post the decision made by the human investigator about the claim in the database.

GET /api/cases/{id}/chat : retrieve all chat happened for specified case id

POST /api/cases/{id}/chat : post endpoint to send message to llm and store it in the db for persistence

GET /docs : OpenAPI (Swagger) documentation

- **Operability:** configuration via .env, ingestion is idempotent, logs progress per case, and is also exposed as a CLI (python scripts/ingest.py) for retries.

5. Trade-offs and risks

Decision	Chosen	Alternative	Trade-off accepted
Reasoning design	Single grounded LLM call, structured output	Multi-agent / tool-using agents	Less “impressive”, but cheaper, faster, easier to audit, and no extra evidence exists for agents to fetch
Call granularity	10 cases per call	One call per case	5× fewer calls (free-tier quota, latency), risk that one bad response affects 10 cases, mitigated by per-batch commit and case_id tagging
Who assigns risk	The LLM, from the raw signals	Deterministic weighted score + LLM narrative	Richer reasoning about combinations of signals, at the cost of run-to-run consistency, a hybrid is the recommended next step
When triage runs	Synchronously at startup	Background worker + job queue	Zero infrastructure, but first boot blocks ~20–40 s and would not scale
Model	Gemini Flash-class, temperature 0	Larger / self-hosted model	Low cost and latency, temperature 0 is not a determinism guarantee
Chat Persistence	Langchain-postgres message history in the same postgres instance	Owned case_messages table with FK to claims, async pool, capped history window	Five-line integration that slots into the langchain message pipeline and keeps one database in return a UUID based session key with no foreign key , a single shared connection and full transcript sent on each turn

Risks Associated.

- **Accuracy & consistency:** no labelled ground truth exists, so the risk level is judgement, not a validated score, identical inputs can drift slightly between runs.
- **Trust & automation bias:** a confident-sounding overview can be accepted too readily hence the confidence badge and cited drivers.
- **Cost & latency:** negligible at 50 cases, at thousands per day the startup-time design must become a queue.
- **Abuse:** chat input reaches the model directly, so prompt-injection or off-topic use is possible, today it is constrained only by the system prompt.
- **Data:** synthetic today, PII tomorrow nothing yet minimises what is sent to a third-party model.

6. Future RoadMap:

1. **Asynchronous ingestion.** Move triage into a background worker (Celery/RQ on Redis) so scoring never blocks the API. Each claim is a retryable job, batched to minimise LLM calls and rate-limited to stay under provider quotas.
2. **Evaluation loop and observability.** Treat every Escalate/Dismiss as a label, track AI–human agreement, precision on escalations and time-to-decision, and run a fixed regression set before any prompt or model change. Trace every LLM call (tokens, latency, cost, model and prompt version) with alerts on failures and spend.
3. **Multi-user readiness.** Authentication and roles, per-team queues, case assignment so two investigators never work the same case, optimistic locking on decisions, and a pooled database connection in place of the single shared one used for chat history.
4. **Guardrails and model lifecycle.** Validate that every cited key driver names a real field and value, re-ask on drift, screen chat input for prompt injection, version prompts, and abstract the provider for fallback and cheaper-model tiering.
5. **Performance and compliance.** Server-side pagination, filtering and search with indexes on risk and status, streamed chat responses for lower perceived latency, encryption at rest and in transit with PII minimised before any third-party model call, plus retention and data-residency policy.

7. Conclusion

The prototype delivers the full loop the brief asked for : ingest, reason, assess before a human looks, then let the human drill in, question, decide and annotate with the AI grounded by construction rather than by hope. The most important product decisions were to make **confidence and evidence first-class outputs**, to keep the **human as the only actor who can change a case's status**, and to spend the limited time polishing one narrow path end to end. The trade-offs are deliberate and reversible: the batch-and-commit pipeline, typed schema and structured outputs are exactly the seams where a job queue, hybrid scorer and evaluation loop plug in next.